Лекція №6 (2 години). Форми, валідація та збір даних у HTML5.

План лекції

1. Роль веб-форм та їх базовий синтаксис.
2. Основні елементи форм (`<input>` , `<textarea>` , `<select>` , `<button>`).
3. Нові типи полів вводу (Input Types) в HTML5.
4. Атрибути елементів управління формами.
5. Клієнтська валідація даних в HTML5 (Client-side Validation).
6. Семантика форм та доступність (Accessibility).

Перелік умовних скорочень

- **HTTP** (HyperText Transfer Protocol) — протокол передачі гіпертексту.
- **URL** (Uniform Resource Locator) — уніфікований покажчик ресурсу.
- **UI** (User Interface) — користувацький інтерфейс.
- **UX** (User Experience) — користувацький досвід.
- **Regex** (Regular Expression) — регулярний вираз.

Вступ

Всесвітня павутина починалася як інструмент для обміну статичними документами. Проте дуже скоро виникла потреба в двосторонній комунікації — можливості для користувача не лише читати інформацію, але й передавати власні дані на сервер. Для цього і були створені **веб-форми**.

Сьогодні форми є критично важливим компонентом будь-якого веб-додатка. Ми використовуємо їх скрізь: для реєстрації чи авторизації (введення логіна і пароля), написання коментарів чи повідомлень у чаті, оформлення покупок в інтернет-магазині, завантаження файлів чи навіть використання рядка пошуку в Google.

З розвитком стандарту HTML5 робота з формами зазнала значних покращень. З'явилися нові типи полів, які підказують мобільним пристроям, яку саме клавіатуру відкрити (наприклад, цифрову для введення номера телефону), а також вбудовані механізми перевірки правильності введених даних (валідації) без необхідності писати складний код на JavaScript.

Завдання розробника — не просто розмістити поля для вводу, але й зробити процес заповнення форми максимально зручним, інтуїтивно зрозумілим, і таким, що страхує користувача від помилок.

1. Роль веб-форм та їх базовий синтаксис

Будь-яка форма на сторінці починається і закінчується парним тегом `<form>`. Всередині цього тегу розміщуються різноманітні елементи управління — поля вводу, чекбокси, списки та кнопки відправки.

Тег `<form>` виступає як "контейнер", що збирає всі дані зі своїх внутрішніх елементів і готує їх до відправки на сервер.

Основні атрибути тегу `<form>`:

- **`action`**: Визначає URL-адресу сервера (або скрипта обробника), куди будуть відправлені дані форми. Наприклад, `action="/submit-registration"`. Якщо атрибут відсутній, дані будуть відправлені на поточну URL-адресу сторінки.
- **`method`**: Визначає HTTP-метод, який буде використано для відправки даних. Два найпопулярніших:
  - `GET`: Дані форми приєднуються безпосередньо до URL (наприклад, `?query=cats&sort=asc`). Використовується для безпечних запитів, що не змінюють стан сервера (пошук, фільтрація). Не підходить для передачі паролів чи великих обсягів даних.
  - `POST`: Дані передаються в тілі HTTP-запиту, приховано від очей користувача. Використовується для відправки конфіденційної інформації (паролі), завантаження файлів або коли дані призводять до створення/зміни записів на сервері (реєстрація, оформлення замовлення).

Приклад базової структури:

```
<form action="/login" method="POST">
  <!-- Тут будуть розміщені елементи форми -->
</form>
```

2. Основні елементи форм

HTML пропонує багатий арсенал елементів (віджетів) для взаємодії з користувачем.

2.1. Елемент `<input>`

Це найуніверсальніший і найчастіше використовуваний елемент у формах. Його зовнішній вигляд та поведінка кардинально змінюються залежно від значення атрибута `type`. За замовчуванням `type="text"`. Це порожній (одинарний) тег: він не вимагає закриваючого тегу.

Ключові стандартні типи:

- `type="text"`: Звичайне однорядкове текстове поле.
- `type="password"`: Поле для введення пароля (символи маскуються крапками або зірочками).
- `type="radio"`: Перемикач. Використовується в групах, коли користувач має обрати *тільки один* варіант із декількох (наприклад, вибір статі). Усі radio-кнопки однієї групи повинні мати однаковий атрибут `name`.
- `type="checkbox"`: Прапорець. Використовується, коли користувач може обрати одразу кілька варіантів (наприклад, додаткові опції замовлення), або для згоди ("Я погоджуюсь з умовами").

2.2. Елемент `<textarea>`

Якщо користувачу потрібно ввести багато тексту (кілька абзаців), наприклад, коментар або відгук, замість `<input>` використовують багаторядкове текстове поле `<textarea>`. Це парний тег. Текст, розміщений між відкриваючим і закриваючим тегами, стане початковим значенням. Розмір можна задати атрибутами `rows` (кількість видимих рядків) та `cols` (кількість видимих символів у рядку), але на практиці розмір частіше контролюють через CSS.

```
<textarea
  name="message"
  rows="5"
  placeholder="Введіть ваше повідомлення..."
></textarea>
```

2.3. Елемент `<select>` (Випадний список)

Дозволяє створити випадний список варіантів. Сам список створюється тегом `<select>`, а кожен варіант вибору всередині нього визначається тегом `<option>`. Значення, яке реально піде на сервер при виборі опції, задається в атрибуті `value` тегу `<option>`, в той час як текст між тегами `<option>` та `</option>` — це те, що бачить користувач.

```
<select name="city">
  <option value="kyiv">Київ</option>
```

```
  <option value="lviv">Львів</option>
  <option value="kharkiv">Харків</option>
</select>
```

2.4. Елемент `<button>`

Кнопка може використовуватися як всередині, так і поза формою. Всередині форми вона зазвичай слугує для її відправки. Атрибут `type` для кнопки принципово важливий:

- `type="submit"` (за замовчуванням у формі): Натискання призводить до валідації та відправки даних форми на сервер.
- `type="reset"` : Скидає всі поля форми до їх початкових значень (рідко використовується на практиці сьогодні, бо часто призводить до випадкової втрати даних користувачем).
- `type="button"` : Звичайна кнопка. Натискання *не здійснює* жодної стандартної дії. Використовується, коли до кнопки за допомогою JavaScript прив'язується якась кастомна логіка.

3. Нові типи полів вводу (Input Types) в HTML5

HTML5 приніс велику кількість нових значень для атрибута `type` елемента `<input>`. Вони призначені для більш спеціалізованого введення даних і мають дві значні переваги:

1. **Покращення UX на мобільних пристроях:** Залежно від типу поля, смартфон чи планшет покаже користувачу оптимальну віртуальну клавіатуру (наприклад, лише цифри, або клавіатуру з готовою кнопкою ".com" та символом "@").
2. **Вбудована базова валідація браузером:** Браузер сам перевіряє, чи відповідає введений текст певному формату. Якщо браузер старий і не розпізнає новий `type`, він безпечно зведе його до базового `type="text"`.

Основні HTML5 типи:

- `type="email"` : Очікується електронна адреса (повинна містити `@` та домен).
- `type="url"` : Очікується абсолютна URL-адреса (наприклад, `https://google.com`).
- `type="tel"` : Для номерів телефонів (як правило, викликає спеціальну цифрову клавіатуру на телефонах, але формату номерів не перевіряє суворо, оскільки в різних країнах вони відрізняються).
- `type="number"` : Для введення чисел. Включає стрілочки (спінер) для збільшення/зменшення значення. Дозволяє обмежити ввід атрибутами `min`, `max` та встановити крок змін `step`.
- `type="range"` : Створює повзунок (слайдер) для вибору числа із заданого діапазону.

- `type="date"` / `time` / `datetime-local` : Відкривають зручні нативні календарі (date pickers) та селектори часу операційної системи для вибору дати і часу без потреби в JavaScript-бібліотеках.
- `type="color"` : Викликає системну палітру вибору кольору.
- `type="file"` : Дозволяє обрати файл (або декілька, з атрибутом `multiple`) з файлової системи комп'ютера.

4. Загальні атрибути елементів управління формами

Окрім `type` , існують важливі атрибути, які керують поведінкою полів вводу:

- `name` : **Критично важливий атрибут.** Він задає ім'я змінної, під яким дані цього конкретного поля будуть відправлені на сервер. Наприклад, якщо `name="username"` , а користувач ввів "Ivan", сервер отримає пару `username=Ivan` . Елемент без атрибута `name` не бере участі у відправленні форми.
- `value` : Задає початкове значення для поля (те, що буде написано в ньому при завантаженні). Для елементів `type="radio"` чи `type="checkbox"` це значення відправиться на сервер, якщо цей елемент буде позначений.
- `placeholder` : Підказка (сірий текст), яка відображається всередині текстового поля (`text` , `email` , `password` тощо), коли воно порожнє. Вона зникає, як тільки користувач починає набирати текст. Не замінює повноцінного тегу `<label>` .
- `disabled` : Робить поле недоступним для редагування і кліків. Воно виглядає сірим (неактивним). **Важливо:** Дані з disabled-полів *не відправляються* на сервер.
- `readonly` : Схоже на `disabled` , користувач не може змінити значення, але поле може отримувати фокус, текст можна виділяти і копіювати, і **головне** — дані з нього будуть відправлені з формою.

5. Клієнтська валідація даних в HTML5 (Client-side Validation)

Валідація — це процес перевірки того, чи дані, які ввів користувач, відповідають вимогам та очікуваному формату, *до* того, як ці дані будуть прийняті та оброблені.

Формально, справжня, безпечна валідація має проводитися виключно на **сервері (Back-end)**, оскільки будь-яку перевірку в браузері зловмисник може обійти. Проте, **клієнтська (браузерна)** валідація є життєво необхідною для хорошого UX (User Experience). Вона дозволяє миттєво повідомити користувача про помилки (скажімо, забутий символ "@" в email або занадто короткий пароль), не змушуючи його чекати відповіді сервера після кожного кліку.

Ранні веб-розробники писали багато коду на JavaScript (JS) лише для обробки базових перевірок. HTML5 значно спростив це завдання, ввівши декларативну (вбудовану) перевірку за допомогою спеціальних атрибутів.

Коли користувач натискає кнопку `<button type="submit">`, браузер автоматично перевіряє поля. Якщо якісь дані введені некоректно, браузер скасовує відправку форми і показує біля помилкового поля стандартизоване повідомлення-підказку (наприклад, "Будь ласка, заповніть це поле").

Ключові атрибути для HTML5-валідації:

- `required`: Робить поле обов'язковим. Без його заповнення браузер не дозволить відправити форму.
- `minlength` та `maxlength`: Для текстових полів. Визначають мінімальну і максимальну дозволену кількість символів (включно з пробілами).
- `min` та `max`: Для числових полів (`type="number"`, `type="range"`) або дат (`type="date"`). Визначають найменше і найбільше допустиме значення.
- `pattern`: Найпотужніший інструмент валідації. Дозволяє перевірити введене значення на відповідність регулярному виразу (Regular Expression). Наприклад, можна змусити вводити тільки цифри, або перевірити складний формат поштового індексу.

Приклад використання патерну: Цей input вимагатиме ввести рівно три великі літери англійського алфавіту:

```
<input
  type="text"
  name="countryCode"
  pattern="[A-Z]{3}"
  title="Тільки 3 великі латинські літери"
/>
```

(Атрибут `title` часто використовується разом з `pattern`, щоб пояснити користувачу, який саме формат від нього очікується, коли з'явиться віконце помилки).

Розробники можуть стилізувати елементи в CSS залежно від стану їх валідності за допомогою псевдокласів `:valid` та `:invalid` (наприклад, додавати зелену рамку правильним полям, і червону — тим, що містять помилку).

6. Семантика форм та доступність (Accessibility)

Форми — це часто бар'єр для користувачів з обмеженими можливостями (наприклад, незрячі користувачі, які використовують програми зчитування екрану — Screen Readers). Створення правильної структури за допомогою семантичних тегів вирішує більшість проблем.

6.1. Тег `<label>`

Він визначає підпис до елемента форми. Це не просто візуальний текст. `<label>` створює програмний зв'язок між текстом та відповідним `<input>`. Коли екранний диктор натрапляє на поле, він читає текст з його `<label>`. З точки зору UI це також корисно: клік мишкою по тексту `<label>` автоматично встановлює курсор (фокус) всередину відповідного `<input>` або перемикає чекбокс.

Зв'язування відбувається через атрибут `for` на `<label>`, який має точно співпадати зі значенням атрибуту `id` на `<input>`.

```
<label for="user-email">Електронна пошта:</label>
<input type="email" id="user-email" name="email" />

<!-- Інший спосіб (вкладеність, без атрибута for) -->
<label>
  <input type="checkbox" name="subscribe" /> Підписатися на новини
</label>
```

6.2. Групування `<fieldset>` та `<legend>`

Коли форма велика, її зручно (і семантично правильно) ділити на логічні частини: наприклад, "Особисті дані", "Адреса доставки", "Метод оплати".

- `<fieldset>`: Групує кілька елементів керування всередині форми. Зазвичай браузер малює навколо групи тонку рамку.
- `<legend>`: Задає заголовок (назву) для групи `<fieldset>`, розміщуючись прямо на її верхній рамці.

```
<form>
  <fieldset>
    <legend>Налаштування сповіщень</legend>
    <label><input type="radio" name="notify" value="email" /> Email</label
    ><br />
    <label><input type="radio" name="notify" value="sms" /> SMS</label>
  </fieldset>
</form>
```

Висновки

1. Веб-форми — це головний інтерфейс для збору та відправки інформації від клієнта до сервера (тег `<form>`, атрибути `action` і `method`).
2. Елемент `<input>` є універсальним інструментом. Змінюючи його тип (`type`), ми змінюємо його поведінку (текст, чекбокси, радіо-кнопки тощо).
3. Впровадження нових Input Types в HTML5 (`email`, `date`, `number`) суттєво покращило взаємодію з мобільними пристроями шляхом відображення оптимізованих клавіатур та інтерфейсів.
4. Клієнтська валідація через атрибути HTML (`required`, `minlength`, `pattern`) дозволяє без написання скриптів перевіряти введені дані і показувати повідомлення про помилки, підвищуючи загальний UX. Втім, вона ніколи не повинна заміняти перевірку на сервері.
5. Використання елементів `<label>`, `<fieldset>`, та `<legend>` забезпечує високий рівень доступності (Accessibility), роблячи форми зручними для користувачів зі сліпотою чи іншими вадами.

Джерела

1. MDN Web Docs: HTML forms guide
2. MDN Web Docs: Client-side form validation
3. W3C Web Accessibility Initiative (WAI): Forms Tutorial
4. HTML5 Pattern (html5pattern.com) - бібліотека регулярних виразів для атрибуту pattern.

Запитання для самоперевірки

1. Для чого призначений атрибут `action` у тегу `<form>`?
2. У чому семантична та практична різниця між HTTP-методами `GET` та `POST` при відправці форми?
3. Чим відрізняється поведінка групи елементів `type="radio"` від групи елементів `type="checkbox"`? Завдяки якому атрибуту групуються радіо-кнопки?
4. Опишіть призначення атрибута `name` на елементі вводу. Що станеться, якщо його пропустити?
5. Наведіть щонайменше 2 переваги використання нових типів вводу HTML5 (наприклад, `type="email"`, `type="date"`).
6. Чому важливо використовувати тег `<label>`? Як саме його "пов'язують" із текстовим полем?
7. У чому різниця між атрибутами стану `disabled` та `readonly`?
8. Що таке клієнтська валідація? Чому, якщо вона є, все одно необхідна валідація даних на бекенді (сервері)?

9. Який атрибут у HTML5 застосовується для перевірки значення поля за допомогою регулярного виразу (regex)?